EXPERIMENT 07

Component-Based Web Application using React JS

Date of Performance: 13/10/2026
DIV/ Batch No: A / 2
Student Name: Kehan Desai
Roll No: 16010125050

Course Outcome: CO4 – Understanding and implementing web application using React.

Aim: To develop a component-based web application using React JS by implementing JSX, functional components, props, state and event handling for case study on……..

Objectives

Create and run a basic React application using Node.js and npm.
Understand JSX as an XML-like syntax extension used inside JavaScript for describing React UI.
Develop reusable functional components.
Pass data between components using props.
Manage changing UI data using state and useState().
Handle user interaction using React event handlers.

NOTE: Refer to the "Case Study you selected" and complete the tasks in React. This write-up uses Student Information Application as a sample.

Software Requirements

Node.js and npm
Visual Studio Code
Google Chrome or another modern browser
Command Prompt/PowerShell

Theory

React and Existing HTML/CSS/JavaScript Knowledge

HTML provides structure, CSS provides presentation, and JavaScript provides programming logic and browser interaction. React organizes UI development around reusable components, JSX, props and state.

JSX and XML Correlation

JSX stands for JavaScript XML. It looks similar to XML/HTML because it uses angle-bracket tags, but JSX is not XML. It is a syntax extension used to describe React UI inside JavaScript and is transformed by the build tooling.

```
const name = "Riya";
function App() { return <h1>Hello {name}</h1>; }
```

HTML: class="box" → JSX: className="box".
HTML onclick="..." → JSX onClick={...}.
JSX permits JavaScript expressions inside { }.
XML represents structured data using tags; JSX describes UI.
JSON represents data using key-value pairs and arrays.

Functional Components

A functional component is a JavaScript function that returns JSX. Components divide a UI into reusable units.

```
function Student() {
  return <h2>Student Details</h2>;
}
```

Props

Props are read-only inputs supplied by a parent component to a child component.

```
function Student(props) {
  return <p>{props.name} - {props.course}</p>;
}

<Student name="Riya" course="B.Tech" />
```

State and Event Handling

State represents data that can change during application execution. Updating state causes React to render the relevant UI again.

```
const [age, setAge] = useState(20);
<button onClick={() => setAge(age + 1)}>Increase Age</button>
```

Problem Statement

Develop a Student Information application that displays student details using a reusable Student component. The application must accept student information through props and provide an interactive control that changes the displayed age using React state and event handling.

Tasks to be Completed:

1. Verify Node.js and npm installation.
2. Create a React application named react-student.
3. Run the application and identify the public and src folders.
4. Modify App.js to display a Student Information heading.
5. Create Student.js as a reusable functional component.
6. Pass name, course and age from App.js to Student.js using props.

7. Display the received information using JSX.
8. Add a state variable for age using useState().
9. Create an Increase Age button using onClick.
10. Apply basic CSS using App.css.
11. Run and test the application in the browser.
12. Capture screenshots of the final output and important source code.

Procedure

13. Open Command Prompt and navigate to the practical folder, for example D:\React.
14. Run: npx create-react-app react-student
15. Run: cd react-student
16. Run: npm start
17. Open the project in VS Code.
18. Create Student.js inside src.
19. Write the Student component and export it.
20. Import Student into App.js and supply props.
21. Add useState and the Increase Age event.
22. Save and test the application.

Suggested Components

Keep the application very simple:

e.g. Student_Application

```
└── src
    ├── App.jsx
    ├── StudentInfo.jsx
    ├── Result.jsx
    ├── App.css
    └── main.jsx
```

Program

// Student.js

```
import { useState } from "react";

function Student({ name, course, initialAge }) {
  const [age, setAge] = useState(initialAge);
  return (
    <div className="card">
      <h2>Student Details</h2>
      <p>Name: {name}</p>
```

```
      <p>Course: {course}</p>
      <p>Age: {age}</p>
      <button onClick={() => setAge(age + 1)}>
        Increase Age
      </button>
    </div>
  );
}
export default Student;
```

// App.js

```
import Student from "./Student";
import "./App.css";

function App() {
  return (
    <div>
      <h1>Student Information</h1>
      <Student name="Riya" course="B.Tech" initialAge={20} />
    </div>
  );
}
export default App;
```

/* App.css */

```
.card { border: 1px solid #999; padding: 20px; width: 300px; margin: 20px auto; }
button { padding: 8px 15px; }
```

Program / Code

src/App.js

```
import HeatwaveMonitor from "./HeatwaveMonitor";
import "./App.css";

function App() {
  return (
    <main className="app">
      <header className="page-header">
        <p className="eyebrow">AGRICULTURE | CLIMATE INTELLIGENCE</p>
        <h1>Heatwave Monitoring</h1>
        <p>Prediction and Early Warning</p>
      </header>
      <section className="intro">
        <h2>Climate monitoring stations</h2>
        <p>Use the buttons to simulate a temperature change.
          Each station maintains its own state.</p>
      </section>
      <div className="station-grid">
        <HeatwaveMonitor
          location="Mumbai" stationId="AWS001" crop="Maize"
          initialTemperature={42} expectedRise={1} threshold={45}
        />
        <HeatwaveMonitor
          location="Nashik" stationId="AWS002" crop="Wheat"
          initialTemperature={39} expectedRise={2} threshold={45}
        />
      </div>
      <footer>Simulated readings and predictions for a React lab.
        Demo heatwave threshold: 45 °C.</footer>
    </main>
  );
}

export default App;
```

src/EarlyWarning.js

```
function EarlyWarning({ status, tone, threshold }) {
  const messages = {
    normal: "Current and predicted temperatures are below the threshold.",
    watch: "The simulated prediction reaches the threshold. Early warning issued.",
    warning: "The current temperature reaches the threshold. Heatwave alert issued."
  };

  return (
    <section className={`alert ${tone}`} role="status">
      <h3>{status}</h3>
      <p>{messages[tone]}</p>
      <small>Demo alert threshold: {threshold} °C</small>
    </section>
  );
}

export default EarlyWarning;
```

Program / Code

src/HeatwaveMonitor.js

```
import { useState } from "react";
import EarlyWarning from "./EarlyWarning";

function HeatwaveMonitor({ location, stationId, crop,
  initialTemperature, expectedRise, threshold }) {
  const [temperature, setTemperature] = useState(initialTemperature);
  const predictedTemperature = Math.min(60, temperature + expectedRise);
  const status = temperature >= threshold ? "Heatwave Warning"
    : predictedTemperature >= threshold ? "Early Warning" : "Normal";
  const tone = temperature >= threshold ? "warning"
    : predictedTemperature >= threshold ? "watch" : "normal";

  return (
    <article className="station-card" aria-label={`${location} station`}>
      <div className="card-heading">
        <div><p className="station-id">{stationId}</p>
          <h2>{location}</h2></div>
        <span className="crop">Crop: {crop}</span>
      </div>
      <div className="readings" aria-live="polite">
        <div><p>Current temperature</p>
          <strong className="current-temperature">{temperature} °C</strong>
        </div>
        <div><p>Tomorrow (simulated)</p>
          <strong className="predicted-temperature">
            {predictedTemperature} °C
          </strong>
        </div>
      </div>
      <p className="model">Demo model: current + {expectedRise} °C,
        capped at 60 °C.</p>
      <EarlyWarning status={status} tone={tone} threshold={threshold} />
      <div className="controls">
        <button disabled={temperature >= 60}
          onClick={() => setTemperature(value => Math.min(60, value + 1))}>
          Increase Temperature
        </button>
        <button className="secondary" disabled={temperature <= -10}
          onClick={() => setTemperature(value => Math.max(-10, value - 1))}>
          Decrease Temperature
        </button>
        <button className="reset"
          onClick={() => setTemperature(initialTemperature)}>Reset</button>
      </div>
    </article>
  );
}

export default HeatwaveMonitor;
```

Program / Code

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; background: #f1f7f3; color: #183c2b;
  font-family: Arial, sans-serif; line-height: 1.5; }
.app { max-width: 1120px; margin: 30px auto; padding: 0 24px; }
.page-header { background: #216644; color: white;
  padding: 24px 30px; border-radius: 10px; }
.page-header h1 { font-size: 32px; margin: 4px 0; }
.page-header p { margin: 0; }
.eyebrow { font-size: 12px; letter-spacing: 1.5px; font-weight: bold; }
.intro { margin: 22px 0 18px; }
.intro h2 { margin: 0 0 5px; font-size: 21px; }
.intro p { margin: 0; color: #516359; }
.station-grid { display: grid; grid-template-columns: repeat(2, 1fr);
  gap: 20px; }
.station-card { background: white; border: 1px solid #cbded1;
  padding: 24px; border-radius: 10px; }
.card-heading { display: flex; justify-content: space-between;
  align-items: center; gap: 12px; }
.card-heading h2 { margin: 0; font-size: 25px; }
.station-id { margin: 0; color: #516359; font-size: 12px;
  letter-spacing: 1px; }
.crop { background: #edf5ef; padding: 6px 10px;
  border-radius: 5px; font-size: 13px; }
.readings { display: grid; grid-template-columns: 1fr 1fr;
  gap: 18px; margin: 20px 0 0; }
.readings p { margin: 0 0 3px; font-size: 13px; color: #516359; }
.readings strong { font-size: 30px; }
.predicted-temperature { color: #607567; }
.model { font-size: 12px; color: #516359; margin: 10px 0 18px; }
.alert { padding: 14px 16px; border-left: 4px solid;
  border-radius: 4px; min-height: 142px; }
.alert h3 { margin: 0 0 6px; font-size: 18px; }
.alert p { margin: 0 0 6px; font-size: 14px; }
.normal { background: #e7f6eb; border-color: #287440; color: #215a36; }
.watch { background: #fff3d6; border-color: #af7200; color: #714600; }
.warning { background: #ffe9e6; border-color: #b32323; color: #8b2020; }
.controls { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 18px; }
button { padding: 10px 12px; border: 0; border-radius: 4px;
  background: #216644; color: white; font: inherit;
  font-size: 13px; font-weight: bold; cursor: pointer; }
.secondary { background: #e6efe9; color: #183c2b; }
.reset { background: white; color: #216644; border: 1px solid #819b8c; }
button:hover { filter: brightness(.94); }
button:focus-visible { outline: 2px solid #216644; outline-offset: 3px; }
button:disabled { opacity: .5; cursor: not-allowed; }
footer { margin: 18px 0; text-align: center; color: #526258; font-size: 12px; }
@media (max-width: 760px) {
  .station-grid { grid-template-columns: 1fr; }
  .app { margin: 14px auto; padding: 0 12px; }
  .page-header, .station-card { padding: 20px; }
}
```

Supporting Files

src/index.js

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

const root = createRoot(document.getElementById("root"));
root.render(<React.StrictMode><App /></React.StrictMode>);
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="theme-color" content="#216644" />
  <title>Agriculture | Heatwave Monitoring</title>
</head>
<body>
  <noscript>Enable JavaScript to run this React application.</noscript>
  <div id="root"></div>
</body>
</html>
```

package.json - Dependencies and Commands

The complete package.json and package-lock.json are included in the source-code ZIP. The main dependencies and commands are shown below.

```
{
  "name": "react-heatwave",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^19.3.0",
    "react-dom": "^19.3.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}
```

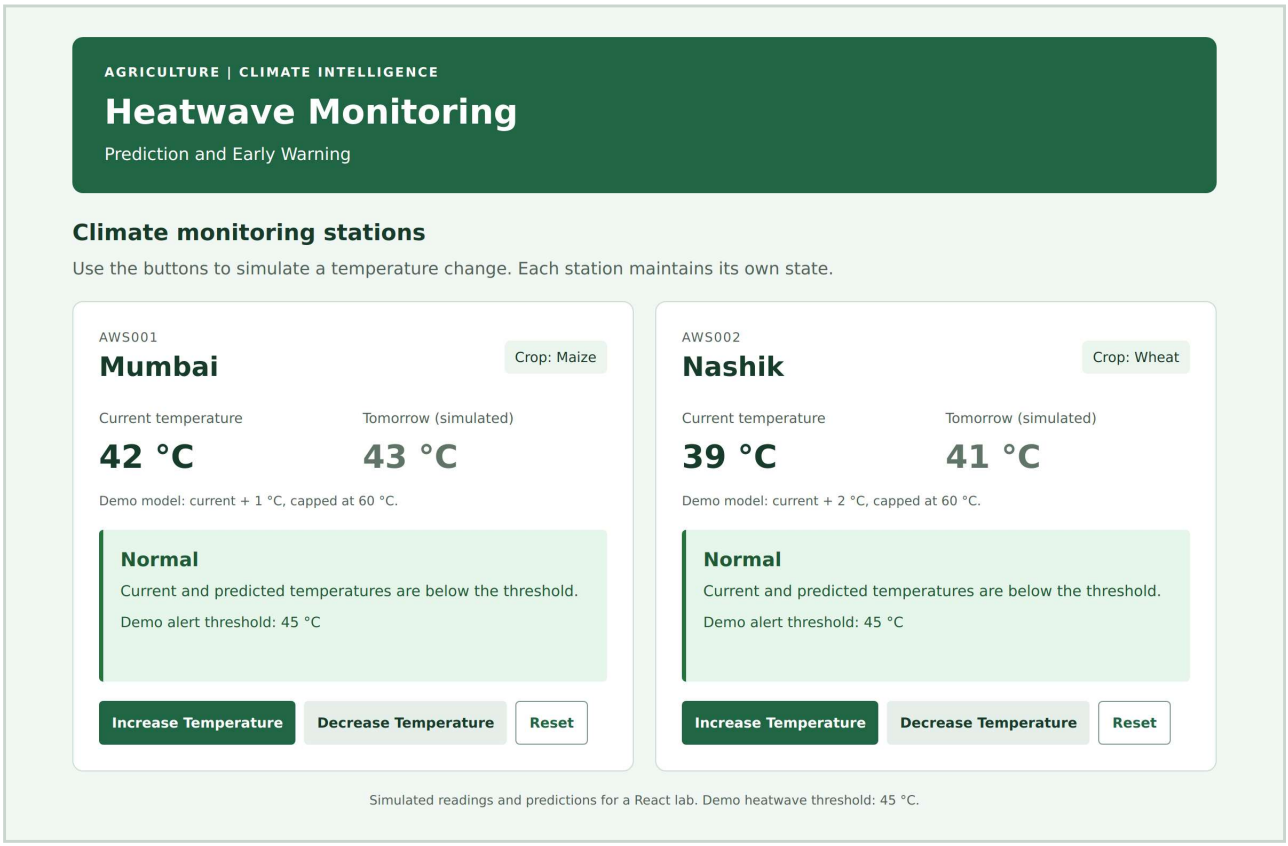
Expected Output

The browser displays two station cards. Each card shows its station ID, location, crop, current temperature, simulated prediction and warning. Increase Temperature adds 1 °C, Decrease Temperature subtracts 1 °C, and Reset restores the initial temperature. Only the selected station changes.

Output / Screenshots

1. Initial output - Normal conditions

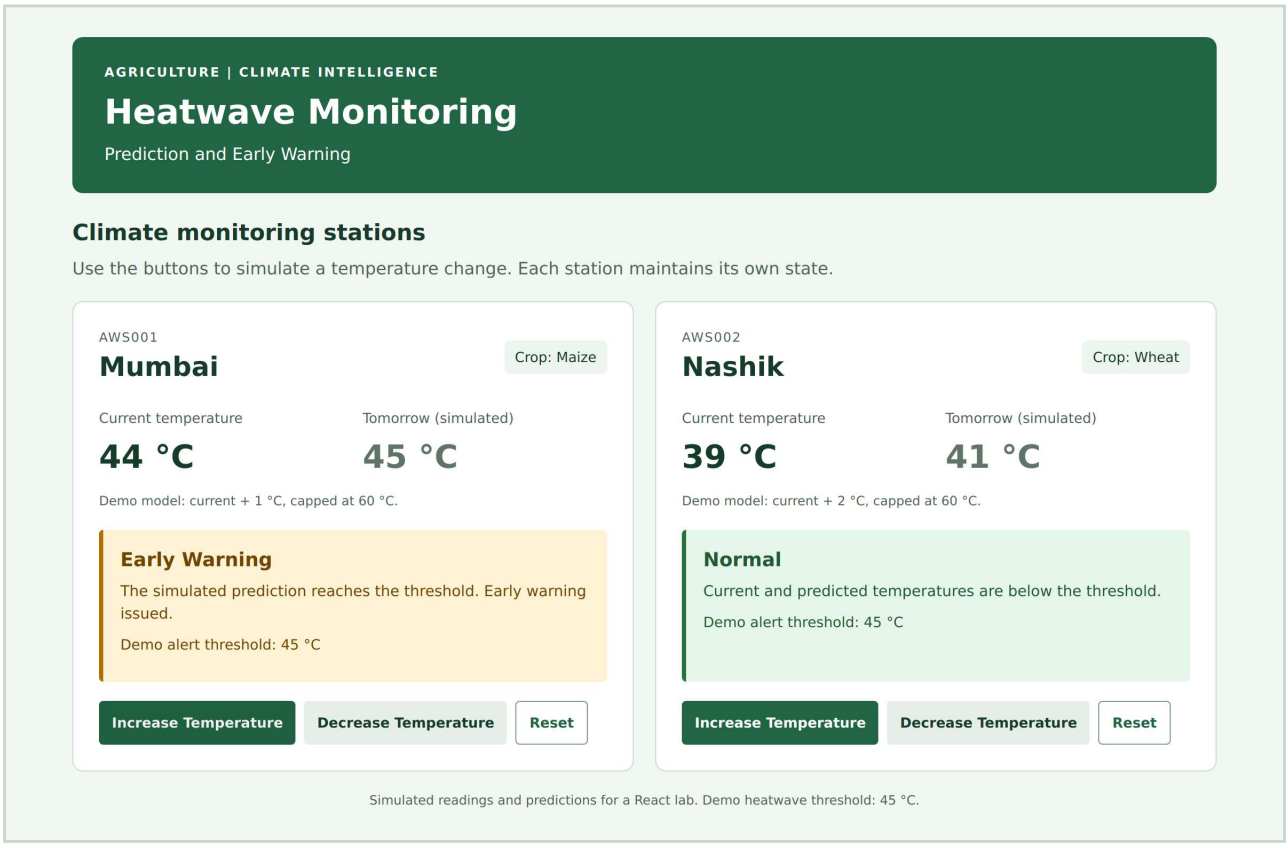
Mumbai starts at 42 °C with a simulated prediction of 43 °C. Nashik starts at 39 °C with a prediction of 41 °C. Both stations display Normal.



Output / Screenshots

2. Early warning after two clicks

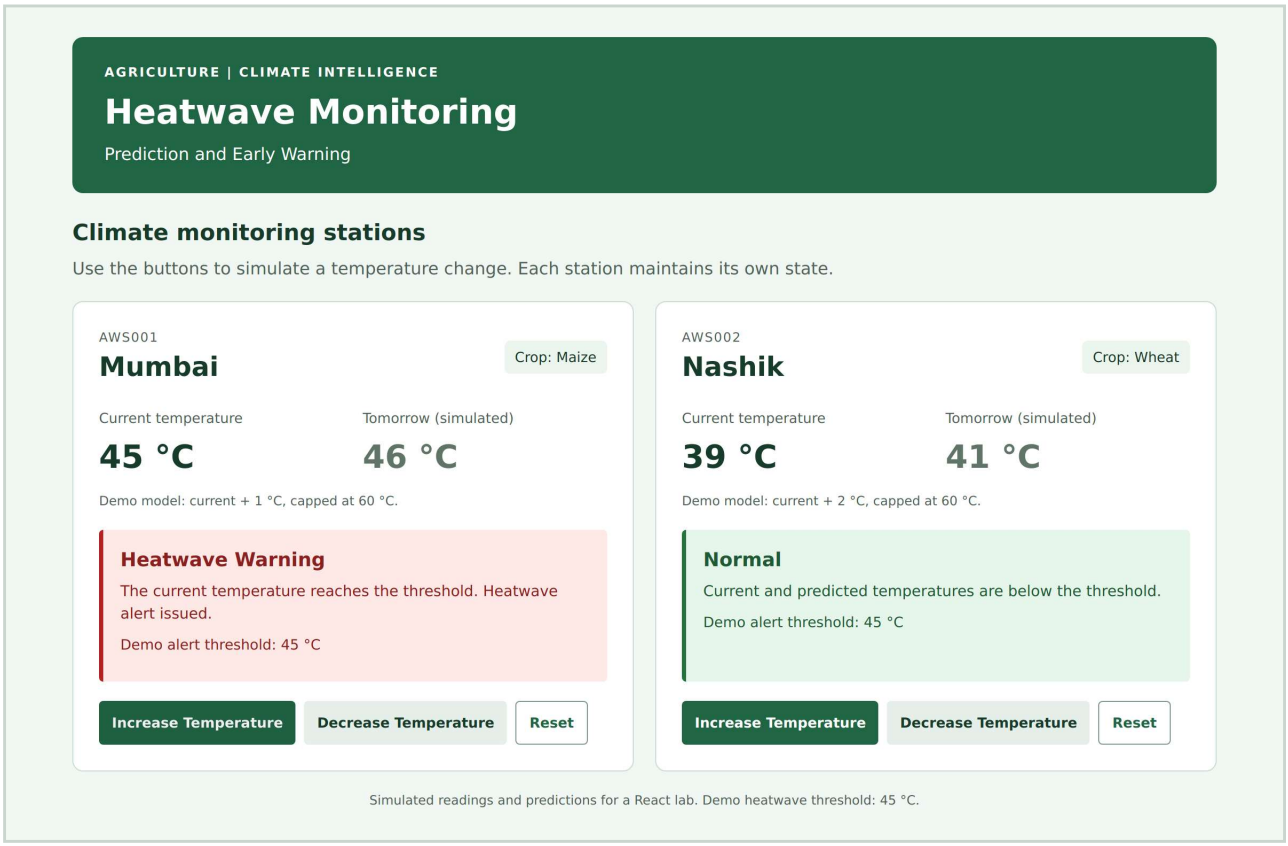
After two clicks on Mumbai's Increase Temperature button, the current value is 44 °C and the prediction reaches 45 °C. The status changes to Early Warning.



Output / Screenshots

3. Heatwave warning at the threshold

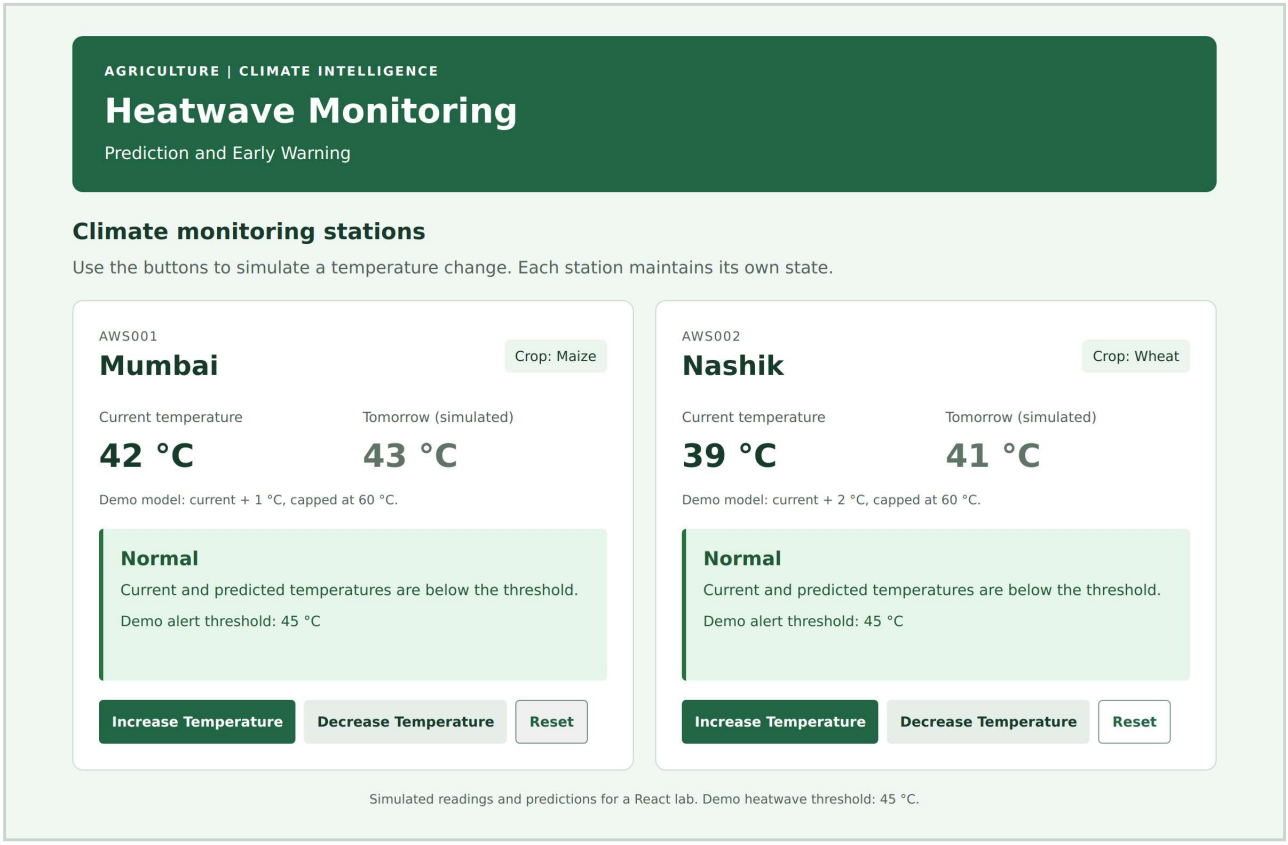
One more click increases Mumbai's current temperature to 45 °C and prediction to 46 °C. Heatwave Warning appears. Nashik remains at 39 °C.



Output / Screenshots

4. Reset restores the initial temperature

Clicking Mumbai's Reset button restores 42 °C, the prediction returns to 43 °C, and the status returns to Normal.



Important Source Code / Screenshot

5. Parent props and reusable warning component

Actual screenshot of a read-only browser source view. App supplies different props to the two station components, and EarlyWarning displays the corresponding message.

Important source code

Read-only browser view of the submitted React source files.

src/App.js

```
import HeatwaveMonitor from "./HeatwaveMonitor";
import "./App.css";

function App() {
  return (
    <main className="app">
      <header className="page-header">
        <p className="eyebrow">AGRICULTURE | CLIMATE INTELLIGENCE</p>
        <h1>Heatwave Monitoring</h1>
        <p>Prediction and Early Warning</p>
      </header>
      <section className="intro">
        <h2>Climate monitoring stations</h2>
        <p>Use the buttons to simulate a temperature change.
          Each station maintains its own state.</p>
      </section>
      <div className="station-grid">
        <HeatwaveMonitor
          location="Mumbai" stationId="AWS001" crop="Maize"
          initialTemperature={42} expectedRise={1} threshold={45}
        />
        <HeatwaveMonitor
          location="Nashik" stationId="AWS002" crop="Wheat"
          initialTemperature={39} expectedRise={2} threshold={45}
        />
      </div>
      <footer>Simulated readings and predictions for a React lab.
        Demo heatwave threshold: 45 °C.</footer>
    </main>
  );
}

export default App;
```

src/EarlyWarning.js

```
function EarlyWarning({ status, tone, threshold }) {
  const messages = {
    normal: "Current and predicted temperatures are below the threshold.",
    watch: "The simulated prediction reaches the threshold. Early warning issued.",
    warning: "The current temperature reaches the threshold. Heatwave alert issued."
  };

  return (
    <section className={`alert ${tone}`} role="status">
      <h3>{status}</h3>
      <p>{messages[tone]}</p>
      <small>Demo alert threshold: {threshold} °C</small>
    </section>
  );
}

export default EarlyWarning;
```

Important Source Code / Screenshot

6. useState, JSX and onClick event handling

Actual screenshot of a read-only browser source view. HeatwaveMonitor stores temperature in state, calculates the prediction, and updates the display through button events.

Important source code

Read-only browser view of the submitted React source files.

src/HeatwaveMonitor.js

```
import { useState } from "react";
import EarlyWarning from "./EarlyWarning";

function HeatwaveMonitor({ location, stationId, crop,
  initialTemperature, expectedRise, threshold }) {
  const [temperature, setTemperature] = useState(initialTemperature);
  const predictedTemperature = Math.min(60, temperature + expectedRise);
  const status = temperature >= threshold ? "Heatwave Warning"
    : predictedTemperature >= threshold ? "Early Warning" : "Normal";
  const tone = temperature >= threshold ? "warning"
    : predictedTemperature >= threshold ? "watch" : "normal";

  return (
    <article className="station-card" aria-label={`${location} station`}>
      <div className="card-heading">
        <div><p className="station-id">{stationId}</p>
          <h2>{location}</h2></div>
        <span className="crop">Crop: {crop}</span>
      </div>
      <div className="readings" aria-live="polite">
        <div><p>Current temperature</p>
          <strong className="current-temperature">{temperature} °C</strong>
        </div>
        <div><p>Tomorrow (simulated)</p>
          <strong className="predicted-temperature">
            {predictedTemperature} °C
          </strong>
        </div>
      </div>
      <p className="model">Demo model: current + {expectedRise} °C,
        capped at 60 °C.</p>
      <EarlyWarning status={status} tone={tone} threshold={threshold} />
      <div className="controls">
        <button disabled={temperature >= 60}
          onClick={() => setTemperature(value => Math.min(60, value + 1))}>
          Increase Temperature
        </button>
        <button className="secondary" disabled={temperature <= -10}
          onClick={() => setTemperature(value => Math.max(-10, value - 1))}>
          Decrease Temperature
        </button>
        <button className="reset"
          onClick={() => setTemperature(initialTemperature)}>Reset</button>
      </div>
    </article>
  );
}

export default HeatwaveMonitor;
```

Testing and Observations

• Initial load: Mumbai 42 °C / prediction 43 °C and Nashik 39 °C / prediction 41 °C; both Normal.

• Two Increase clicks: Mumbai 44 °C / prediction 45 °C; Early Warning displayed.

• Third Increase click: Mumbai 45 °C / prediction 46 °C; Heatwave Warning displayed.

• Decrease: Mumbai returns to 44 °C / prediction 45 °C; Early Warning displayed.

• Reset: Mumbai returns to 42 °C / prediction 43 °C; Normal displayed.

• Component reuse: changing either station leaves the other station's temperature unchanged.

• Limits: Increase is disabled at 60 °C; Decrease is disabled at -10 °C. The prediction is capped at 60 °C.

• The mobile layout has no horizontal overflow. The production build compiled successfully, and no browser script errors were found.

Result

The component-based heatwave monitoring application was developed successfully using JSX, functional components, props, state and event handling. The interface displayed simulated temperature predictions and changed its warning automatically when the temperature state changed.

Conclusion and Discussion

In this experiment, I used React to create a reusable climate-monitoring component for two stations. I passed station details using props and changed temperatures using useState and onClick events. I tested normal conditions, early warning, heatwave warning, decrease and reset. This helped me understand how React updates a component-based interface when its state changes.

Post-Lab Questions and Answers

1. How is JSX different from HTML?
JSX is a JavaScript syntax extension used to describe a React interface. Unlike HTML, it allows JavaScript expressions inside braces, uses className and camelCase event attributes, and requires tags to be closed.

2. What is a functional component?
A functional component is a JavaScript function that returns JSX. It can receive props and use Hooks such as useState to manage changing data.

3. Why is className used instead of class in JSX?
JSX uses JavaScript-style property names. className is the React property used to assign a CSS class; it becomes the HTML class attribute in the rendered output.

4. How is React different from direct DOM manipulation?
In direct DOM manipulation, JavaScript selects and changes elements manually. In React, we describe the UI using components, props and state. React compares the rendered output and updates the necessary DOM elements when data changes.